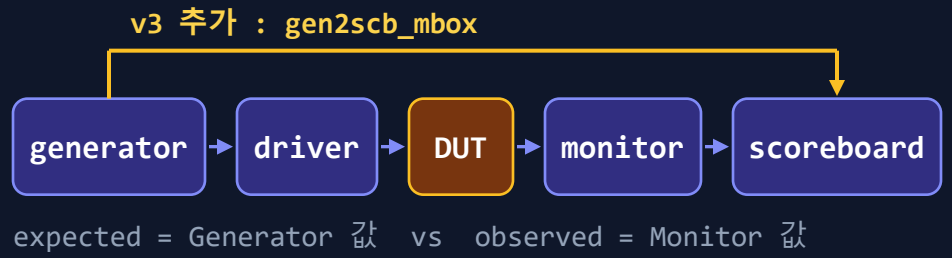
SYSTEMVERILOG · HOMEWORK

# RAM TB v2 vs v3

## 두 검증 모델 차이 비교 분석

tb_ram_v2 · tb_ram_v3 · ram.sv (64 × 8 bit)




expected = Generator 값  vs  observed = Monitor 값

**정안성**

2026. 11

구조

### mailbox 2개 → 3개

gen2scb_mbox 추가, Generator 값을 scoreboard로 직접 전달

판정

### read만 → 매 cycle

판정 건수 v2 138 / 300 · v3 300 / 300

샘플

### posedge → posedge + 1

write 직후 rdata : v2 이전 값 · v3 쓴 값 154 / 154

결과

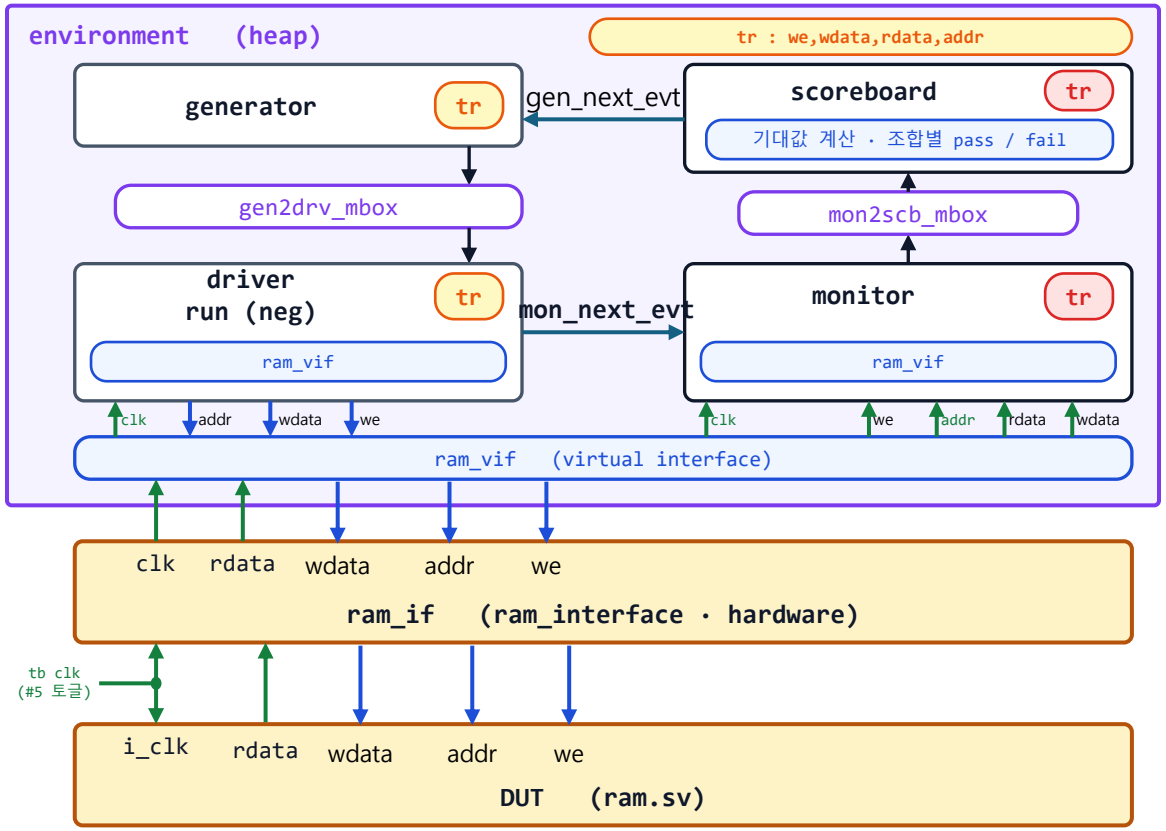
### 64 / 64 주소 확인 · FAIL 0

v2 729 · v3 1143 tr로 전 주소 비교 완료, addr 경로는 보완 필요

# 구조: v3는 generator → scoreboard 경로가 추가된 3-mailbox 구조

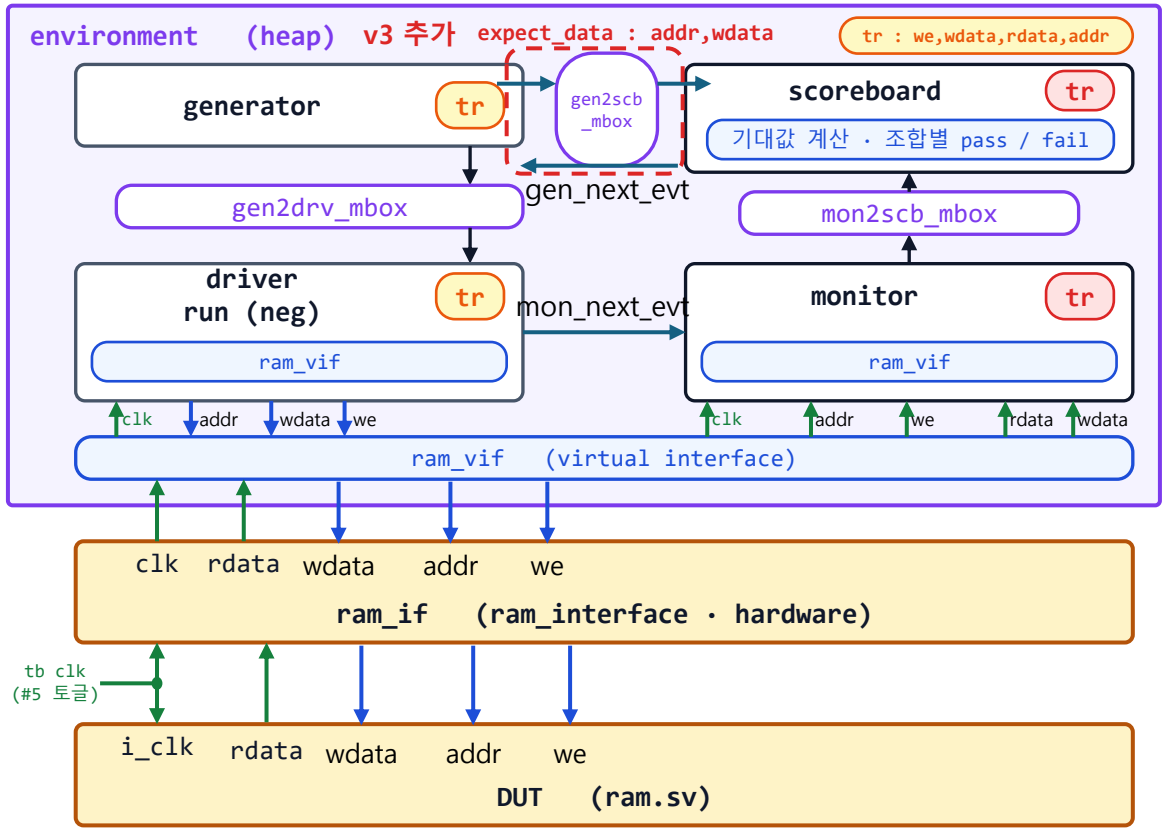
01

**결론** · **v2는 Monitor 값만으로 판정, v3는 Generator 값과 DUT 출력(rdata)을 비교**
· **driver · monitor · interface · DUT 구조는 동일 → 차이는 scoreboard로 들어가는 입력 경로**




**tb_ram_v2.sv**

**mailbox 2개** (gen2drv · mon2scb)
**scoreboard 입력** : Monitor 관측값 1개 → 기대값도 관측값으로 생성



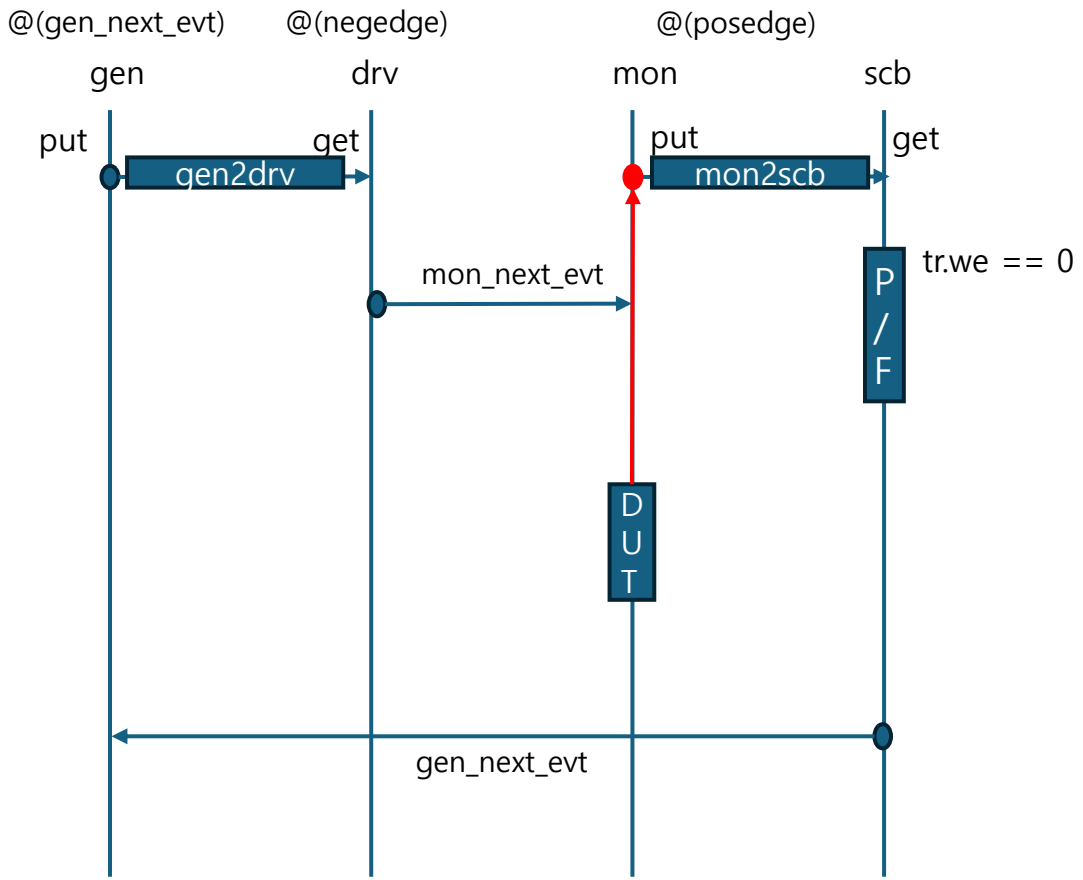

**tb_ram_v3.sv**

**mailbox 3개** (gen2drv · mon2scb · **gen2scb**)
**scoreboard 입력** : Generator wdata (기대값) vs Monitor rdata (DUT 출력)

# 동작 순서: 1 tr = 1 clock 흐름은 같고 판정 입력과 샘플 시점이 다름

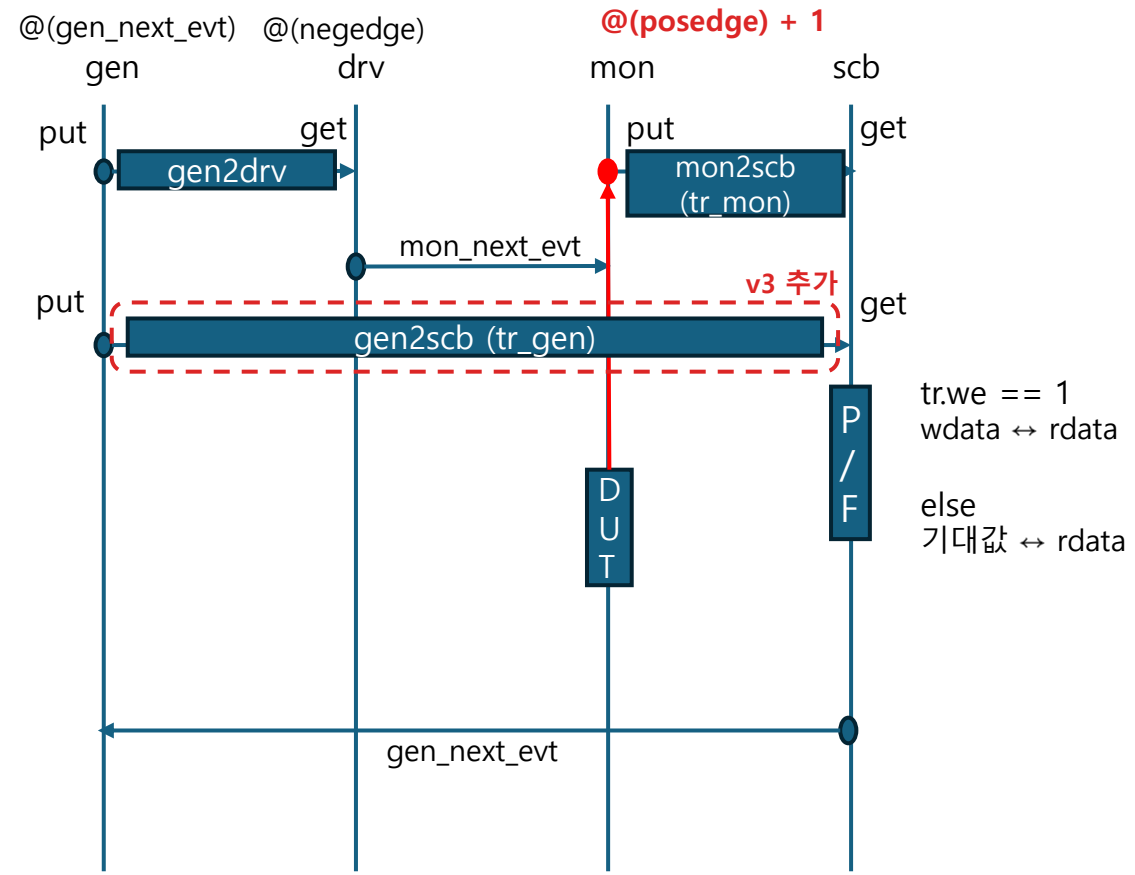
01

**결론**
- **v3는 generator가 tr을 두 mailbox에 put, scoreboard는 두 값을 다 받은 뒤 판정**
- **monitor 샘플 시점 : v2 @(posedge) → v3 @(posedge) + 1 ns**




`tb_ram_v2.sv`

mon 샘플 → scb get → write면 저장, read면 P/F → gen_next_evt




`tb_ram_v3.sv`

scb는 tr_gen · tr_mon 2개 get 후 → write · read 모두 P/F → gen_next_evt

# 차이 요약: 수정 지점은 event · 샘플 · 판정 입력 세 가지

01

결론 · **read 판정식은 같고, write 판정 추가로 판정 건수가 138 → 300건으로 증가**
· **v3의 비용 : mailbox 1개 추가, scoreboard가 두 mailbox를 순서대로 기다림**

| 항목 | tb_ram_v2 | tb_ram_v3 | 의미 |
|---|---|---|---|
| **mailbox** | gen2drv · mon2scb (2개) | **+ gen2scb (3개)** | scoreboard가 입력 의도를 직접 받음 |
| **기대값 출처** | Monitor가 본 wdata | **Generator의 wdata (tr_gen)** | Driver 전달 오류도 비교 대상이 됨 |
| **driver → monitor** | ->> + wait(.triggered) | **-> + @(event)** | 둘 다 300건 누락 없음 |
| **monitor 샘플** | @(posedge) | **@(posedge) + #1** | write 직후 갱신된 rdata 관측 가능 |
| **write (we = 1)** | 기대값 저장만 (판정 없음) | **wdata vs rdata 비교 후 저장** | write 결과를 DUT 출력으로 바로 확인 |
| **read (we = 0)** | expected === rdata | expected === rdata | 동일, DUT 저장값 검증의 핵심 |
| **판정 건수** | 138 / 300 | **300 / 300** | v2는 read 건수만큼 판정 |
| **report** | total + read · write 개수 | **read · write별 pass / fail** | 어느 동작에서 실패했는지 구분 |

# generator: 만든 tr을 driver와 scoreboard 양쪽에 전달

02

**결론** · v3는 gen2scb_mbox를 추가해 Generator 값이 scoreboard로 직접 전달
· randomize · gen_next_evt 대기 흐름은 동일, 전달 경로만 1개 추가

**tb_ram_v2.sv · class generator**

```
class generator;
    transaction tr;
    mailbox #(transaction) gen2drv_mbox;
    event gen_next_evt;

    function new(mailbox #(transaction) gen2drv_mbox,
                 event gen_next_evt);
        this.gen_next_evt =gen_next_evt;
        this.gen2drv_mbox =gen2drv_mbox;
    endfunction

    task run(int run_count);
        repeat(run_count)begin
            tr = new();
            tr.randomize();
            gen2drv_mbox.put(tr);
            tr.debug_print("GEN");
            @(gen_next_evt);
        end
    endtask
endclass
```

**tb_ram_v3.sv · class generator**

```
class generator;
    transaction tr;
    mailbox #(transaction) gen2drv_mbox;
    mailbox #(transaction) gen2scb_mbox;  // 1
    event gen_next_evt;

    function new(mailbox #(transaction) gen2scb_mbox,  // 1
                 mailbox #(transaction) gen2drv_mbox,
                 event gen_next_evt);
        this.gen2scb_mbox =gen2scb_mbox;  // 1
        this.gen_next_evt =gen_next_evt;
        this.gen2drv_mbox =gen2drv_mbox;
    endfunction

    task run(int run_count);
        repeat(run_count)begin
            tr = new();
            tr.randomize();
            gen2drv_mbox.put(tr);
            gen2scb_mbox.put(tr);  // 2
            tr.debug_print("  GEN  ");
            @(gen_next_evt);
        end
    endtask
endclass
```

**1 gen2scb_mbox 추가**

선언 · new 인자 · this 대입 3곳에서 mailbox 1개 추가

**2 put 2회**

gen2drv 직후 같은 시점에 put → scoreboard의 기대값 채널

**같은 handle 공유**

두 mailbox에 같은 tr 객체가 들어감. driver는 읽기만 하므로 값 충돌은 없음

* 구분선 $display, 미사용 변수 enable은 생략

# driver · monitor: 이벤트 방식만 바뀌고 tr 전달 흐름은 동일

02

**결론** · v2 ->> + wait(.triggered) → v3 -> + @(event), 두 방식 모두 300건 누락 없음
· @는 트리거 전에 이미 대기 중이어야 함 → v3 로그에서 MON 16 ns 대기 시작, DRV 20 ns 트리거

**tb_ram_v2.sv · driver · run()**

```
task run();
    forever begin
        @(negedge ram_vif.clk);
        gen2drv_mbox.get(tr);
        tr.debug_print("DRV");
        ram_vif.we = tr.we;
        ram_vif.addr = tr.addr;
        ram_vif.wdata = tr.wdata;
        ->>mon_next_evt; // nba tiggered  혼합해서 못사용함 (1)
    end
endtask
```

**tb_ram_v3.sv · driver · run()**

```
task run();
    forever begin
        @(negedge ram_vif.clk);
        gen2drv_mbox.get(tr);
        tr.debug_print("  DRV  ");
        ram_vif.we = tr.we;
        ram_vif.addr = tr.addr;
        ram_vif.wdata = tr.wdata;
        ->mon_next_evt; // nba tiggered  혼합해서 못사용함 (2)
    end
endtask
```

**tb_ram_v2.sv · monitor · run()**

```
task run();
    forever begin
        wait(mon_next_evt.triggered);    // blockng assignment (1)
        @(posedge ram_vif.clk);
        tr = new();
        tr.rdata = ram_vif.rdata ;
        tr.we    =   ram_vif.we ;
        tr.addr = ram_vif.addr  ;
        tr.wdata=   ram_vif.wdata ;
        mon2scb_mbox.put(tr);
        tr.debug_print("MON");
    end
endtask
```

**tb_ram_v3.sv · monitor · run()**

```
task run();
    forever begin
        @(mon_next_evt);    // blockng assignment (2)
        @(posedge ram_vif.clk);
        #1; (3)
        tr = new();
        tr.rdata = ram_vif.rdata ;
        tr.we    =   ram_vif.we ;
        tr.addr = ram_vif.addr  ;
        tr.wdata=   ram_vif.wdata ;
        mon2scb_mbox.put(tr);
        tr.debug_print("  MON  ");
    end
endtask
```

**1 ->> + wait(.triggered)**

NBA 영역에서 트리거, triggered는 같은 time step 동안 유지 → 경쟁에 강함

**2 -> + @(event)**

Active 영역에서 즉시 트리거. 대기가 먼저 걸려 있어야 수신

**3 #1 추가**

샘플 시점을 1 ns 뒤로 이동 → 다음 장

02

# rdata 샘플링(v3): write 판정에 posedge + 1 ns 샘플 필요

**결론** · **posedge 샘플은 RAM 갱신 전 이전 값을 읽어 정상 write도 FAIL 판정**
· **posedge + 1 ns 샘플은 RAM 갱신 후 값을 읽어 쓴 값과 일치 → PASS**

**tb_ram_v3.sv · scoreboard**
write 판정 (we == 1)

```
if(tr_gen.we)begin

    if(tr_gen.wdata == tr_mon.rdata)begin
            w_pass_count ++;
            t_pass_count ++;
    end
```

generator wdata와 DUT rdata 비교

**샘플 시점 비교**

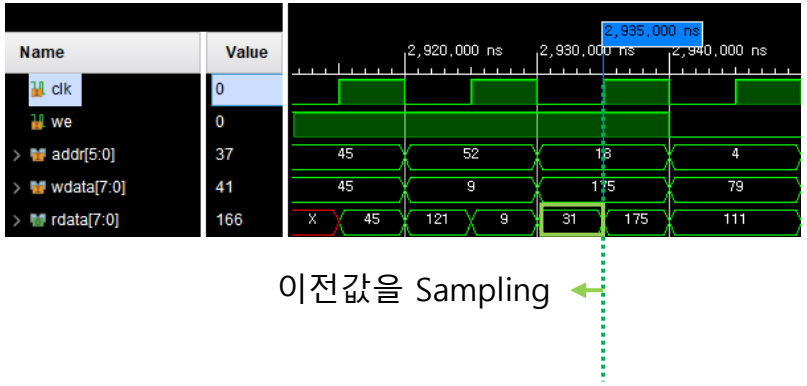
posedge : 2935 ns
rdata 31 ≠ wdata 175 → FAIL
+ 1 ns : 2846 ns
rdata 173 = wdata 173 → PASS

**monitor : @(posedge)**
Active에서 먼저 읽기, RAM 갱신(NBA)은 그 뒤

```
2925ns :[  GEN  ] addr = 18 ,we = 1, wdata = 175 , rdata=   x
2925ns :[SCB-GEN] addr = 18 ,we = 1, wdata = 175 , rdata=   x
2930ns :[  DRV  ] addr = 18 ,we = 1, wdata = 175 , rdata=   x
2935ns :[  MON  ] addr = 18 ,we = 1, wdata = 175 , rdata=  31
2935ns :[SCB-MON] addr = 18 ,we = 1, wdata = 175 , rdata=  31

2935ns  FAIL : tr.rdata  =  31  / expected_rdata = 175
```

drive
monitor

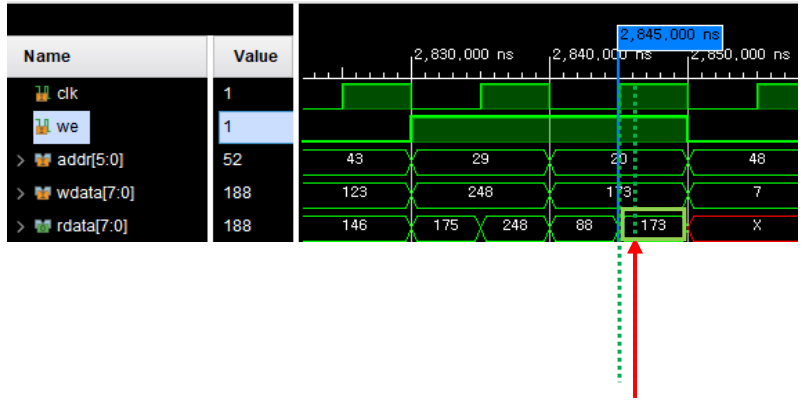



이전값을 Sampling

**monitor : @(posedge) + 1 ns**
RAM 갱신(NBA) 이후 샘플

```
2836ns :[  GEN  ] addr = 20 ,we = 1, wdata = 173 , rdata=   x
2836ns :[SCB-GEN] addr = 20 ,we = 1, wdata = 173 , rdata=   x
2840ns :[  DRV  ] addr = 20 ,we = 1, wdata = 173 , rdata=   x
2846ns :[  MON  ] addr = 20 ,we = 1, wdata = 173 , rdata= 173
2846ns :[SCB-MON] addr = 20 ,we = 1, wdata = 173 , rdata= 173
```

drive
monitor



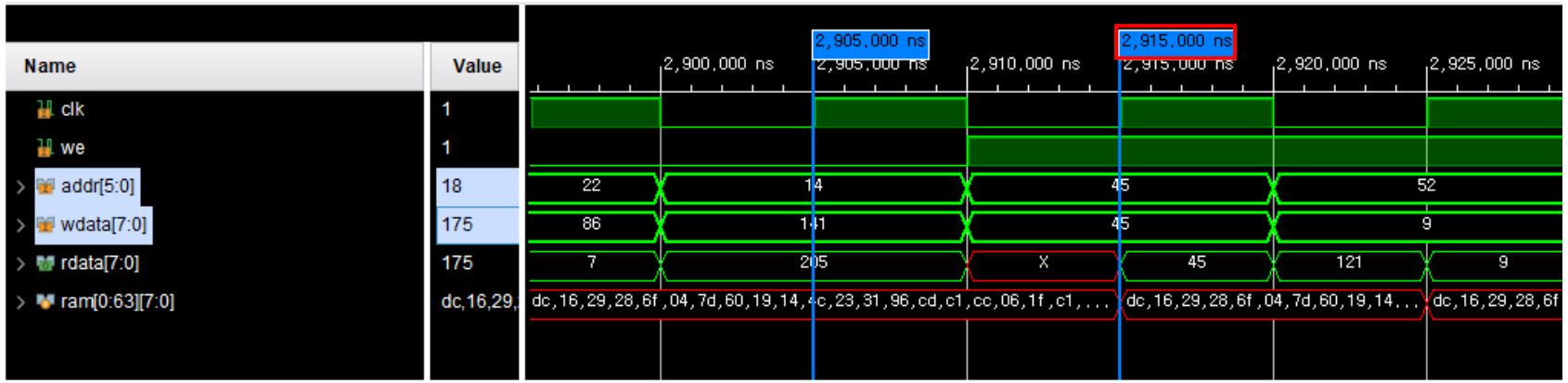

sampling 시점

02

# rdata 샘플링(v2): write 직후 이전 값 샘플, 판정 없어 영향 없음

**결론** · monitor가 RAM 갱신보다 먼저 읽어 write 직후 rdata는 이전 값
· v2 scoreboard는 write에서 rdata를 비교하지 않아 결과에 영향 없음

`tb_ram_v2 · xsim waveform`




**v2 write 판정**

expected 저장만 수행
rdata 비교 없음 → 이전 값이어도 FAIL 미발생

`tb_ram_v2 · xsim log`

```
2905ns :[GEN] addr = 45 ,we = 1, wdata =  45 , rdata=   x
2910ns :[DRV] addr = 45 ,we = 1, wdata =  45 , rdata=   x
2915ns :[MON] addr = 45 ,we = 1, wdata =  45 , rdata=   x
2915ns :[SCB] addr = 45 ,we = 1, wdata =  45 , rdata=   x

2915ns :[GEN] addr = 52 ,we = 1, wdata =   9 , rdata=   x
2920ns :[DRV] addr = 52 ,we = 1, wdata =   9 , rdata=   x
2925ns :[MON] addr = 52 ,we = 1, wdata =   9 , rdata= 121
2925ns :[SCB] addr = 52 ,we = 1, wdata =   9 , rdata= 121
```

**1 2915 ns**

addr 45 write
monitor rdata = X
갱신 후 rdata = 45

**2 2925 ns**

addr 52 write
monitor rdata = 121
갱신 후 rdata = 9

# scoreboard: write에도 판정을 추가해 매 tr마다 pass / fail

02

**결론** · v3는 tr_gen · tr_mon 두 mailbox에서 get → write · read 모두 DUT 출력(rdata)으로 판정
· v2의 write는 expected 저장만 수행, 쓴 값은 이후 read에서 간접 검증

tb_ram_v2.sv · scoreboard · run()

```
task run();
    forever begin
        mon2scb_mbox.get(tr);
        tr.debug_print("SCB");
        total_count ++;

        if(tr.we)begin
            expected_rdata[tr.addr] = tr.wdata;  (1)
            write_count ++;
        end

        else begin
            read_count ++;

            if(expected_rdata[tr.addr] === tr.rdata)begin
                pass_count ++;  (4)
            end

            else begin
                fail_count ++;
                // FAIL $display 3줄 생략
            end
        end

        ->gen_next_evt;
    end
endtask
```

tb_ram_v3.sv · scoreboard · run()

```
task run();
    forever begin
        gen2scb_mbox.get(tr_gen);
        tr_gen.debug_print("SCB-GEN");  (2)
        mon2scb_mbox.get(tr_mon);
        tr_mon.debug_print("SCB-MON");
        total_count ++;

        if(tr_gen.we)begin
            if(tr_gen.wdata == tr_mon.rdata)begin  (3)
                w_pass_count ++;
                t_pass_count ++;
            end
            else begin
                w_fail_count ++;
                t_fail_count ++;
            end
            expected_rdata[tr_gen.addr] = tr_gen.wdata;
            write_count ++;
        end
        else begin
            read_count ++;
            if(expected_rdata[tr_gen.addr] === tr_mon.rdata)begin
                r_pass_count ++;  (4)
                t_pass_count ++;
            end
            else begin
                r_fail_count ++;
                t_fail_count ++;
                // FAIL $display 3줄 생략
            end
        end
        ->gen_next_evt;
    end
endtask
```

**1 v2 write**
관측한 wdata를 기대값으로 저장, 판정 없음

**2 get 2회**
tr_gen → tr_mon 순서로 둘 다 도착해야 진행

**3 write 판정**
Generator wdata와 Monitor rdata를 비교

**4 read 판정**
=== 로 X까지 비교, 두 버전 동일

03

# 트러블 슈팅: 300 tr로는 64개 주소 전체 검증 불가

**결론**

- **문제 : 실제 저장값 비교는 v2 95건 · v3 89건, 나머지 read는 아직 안 쓴 주소를 읽은 X === X**
- **원인 : repeat 횟수 부족, 64개 주소의 확인 여부를 보는 종료 로직 부재**




주소 커버리지 (300 tr)

| 항목 | tb_ram_v2 | tb_ram_v3 |
|---|---|---|
| **write** | 162건 | 154건 |
| **한 번이라도 쓴 주소** | **61 / 64** | **60 / 64** |
| **끝까지 안 쓴 주소** | **33, 38, 48** | **3, 35, 48, 60** |
| **read** | 138건 | 146건 |
| **미기록 주소 read (X === X)** | 43건, 주소 26종 | 57건, 주소 34종 |
| **쓴 값 비교 read** | 95건 (69%) | 89건 (61%) |
| **마지막 미기록 read** | 2585 ns | 2966 ns |

# 해결: 64개 주소 확인 완료를 종료 조건으로 추가

03

**결론** · repeat 횟수 대신 쓴 뒤 read까지 확인한 주소 수로 종료 시점 결정
· v2 · v3 동일 적용, run_count는 5000으로 올린 상한선 역할

tb_ram_v3.sv · scoreboard

```
class scoreboard;
  bit written [0:63];
  bit checked [0:63];
  int checked_count;
  bit done;

else begin

    read_count ++;

    if(written[tr_gen.addr] && !checked[tr_gen.addr])begin
            checked[tr_gen.addr] = 1;
            checked_count ++;
            if(checked_count == 64) done = 1;
    end
```

① ②

tb_ram_v3.sv · environment · run()

```
class environment;

    task run(int run_count);  // argument 선언

        fork
            gen.run(run_count);
            drv.run();
            mon.run();
            scb.run();
            wait(scb.done);  // 64개 주소 확인이 끝나면 join_any 통과 (run_count는 상한선)
        join_any

            scb.report();

            disable fork;

    endtask
```

③

**① 주소별 기록**

written : write된 주소 표시
checked : 쓴 뒤 read로 비교한 주소 표시

**② 완료 판정**

쓴 주소의 첫 read에서만 checked_count 증가
64 도달 시 done = 1

**③ 종료 조건**

wait(scb.done)으로 join_any 통과
run_count 5000은 상한선

03

# 적용 결과: v2 · v3 모두 64개 주소 확인 완료, fail 0

**결론** · **v2 729 tr · v3 1143 tr에서 64개 주소 전부 쓰고 읽어 비교 완료**
· **tr 수 차이는 stimulus 순서 차이, 버전 간 우열 비교 불가**

`tb_ram_v2.sv` · `report`

```
+-------------------------------------------------+
|                V2 TEST  REPORT                  |
+-------------------------------------------------+
|  total test =   729                             |
|  pass count =   346                             |
|  fail count =     0                             |
+-------------------------------------------------+
| read  count =   346                             |
| write count =   383                             |
+-------------------------------------------------+
| addr check  =    64 / 64                        |
| all checked = YES                               |
+-------------------------------------------------+
```

`tb_ram_v3.sv` · `report`

```
+-------------------------------------------------+
|                V3 TEST  REPORT                  |
+-------------------------------------------------+
|  total test =  1143                             |
|  pass count =  1143                             |
|  fail count =     0                             |
+-------------------------------------------------+
| read  count =   574                             |
| read  pass  =   574                             |
| read  fail  =     0                             |
+-------------------------------------------------+
| write  count =   569                            |
| write  pass  =   569                            |
| write  fail  =     0                            |
+-------------------------------------------------+
| addr check  =    64 / 64                        |
| all checked = YES                               |
+-------------------------------------------------+
```

| 구분 | 총 tr (적용 전 → 후) | 쓴 주소 (전 → 후) | fail (전 → 후) |
|---|---|---|---|
| **tb_ram_v2** | 300 → 729 | 61 → 64 | 0 → 0 |
| **tb_ram_v3** | 300 → 1143 | 60 → 64 | 0 → 0 |

### 1 종료 시점

상한선 5000 미도달
64개 주소 확인 시점에 종료

### 2 v2 pass 346건

v2는 read만 판정
pass = read 346건, write 383건은 판정 없음

### 3 fail 0 유지

v2 read 346건
v3 read 574건 · write 569건 전부 PASS

03

# 분석: v3 write는 DUT까지 검증, Driver addr 경로는 미검증

**결론**
- **tr_gen.wdata == tr_mon.rdata로 변경 → 쓰기 안 됨 고장을 154 / 154건 해당 cycle에 검출**
- **v2가 놓친 Driver wdata 오류도 검출, Driver addr 오류는 전 버전 미검출 → addr 비교 필요**

### 판정 범위

| 판정 | 비교식 | 확인 범위 | 쓰기 안 됨 고장 |
|---|---|---|---|
| **v2 · v3 read** | `expected === rdata` | DUT 저장 · 읽기 | 95 · 89건 (read) |
| **이전 v3 write** | `tr_gen.wdata == tr_mon.wdata` | Driver → interface | **0건** |
| **현재 v3 write** | **`tr_gen.wdata == tr_mon.rdata`** | **DUT 저장 즉시** | **154 / 154건** |

### 두 모델 정리

| | v2 | v3 |
|---|---|---|
| **장점** | 구조 단순, mailbox 2개 | 입력 의도와 관측값 비교 |
| **주의** | write 결과는 read 때까지 미확인 | 샘플 타이밍 · 조합 출력 RAM 전제 |
| **적합** | read 위주 기능 확인 | tr 단위로 바로 판정할 때 |

### 개선 항목

**1 addr · we도 비교**

Driver addr 오류 주입 시 전 버전 fail 0
write · read가 같은 틀린 주소를 사용 → gen2scb 값과 비교

```
tr_gen.addr === tr_mon.addr
```

**2 randc we 유지**

매 tr new()로 randc 순환이 끊김
v2 로그 처음 6건 연속 write → 객체 재사용으로 순환 유지

```
tr = new();  // repeat 밖으로 이동
```